Beta Testing Document

for

TrackMate

Version 1.01

Prepared by

Group 5: **Group Name:** Thunderbolts

Hardik Tiwari	230437	hardikt23@iitk.ac.in
Pranjali Deshpande	240336	deshpandep24@iitk.ac.in
Mahi Mittal	240608	mahii24@iitk.ac.in
Mayukh Chowdhury	240336	cmayukh@iitk.ac.in
Palak Bandhu	240719	palakb24@iitk.ac.in
Praveen	240790	praveenas24@iitk.ac.in
Shreshthraj Bhidodiya	240996	sbhidodiya24@iitk.ac.in
Sneha Kumari	241024	snehak24@iitk.ac.in
Virendra Kala	241172	virendrak24@iitk.ac.in
Yash Dabi	241195	yashd24@iitk.ac.in

Course: CS253

Mentor TA: *George T L*

Date: 17/04/2026

CONTENTS II
REVISIONS II

1 INTRODUCTION 1

2 LIST OF BUGS 2

3 OVERALL QUALITY OF THE SOFTWARE 4

APPENDIX A - GROUP LOG 5

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.00	Thunderbolts	The initial draft was prepared	14/04/26
1.01	Thunderbolts	The draft was reviewed and final changes were made, along with formatting optimization	17/04/26

1 Introduction

1.1 Project Overview

TrackMate is an AI-integrated fitness and workout tracking mobile application developed by Group 4 (Earth's Mightiest Dozen (Minus 2)) for the course CS253.
The application is designed to serve as a one-stop solution for gym-goers and fitness enthusiasts by combining nutrition management, workout tracking, and social engagement into a single platform. It allows users to

- log meals and track caloric intake
- record workout sessions with detailed set and rep data
- monitor daily step counts using device sensors, and
- connect with certified trainers for professional guidance

The app also incorporates an LLM-based AI feature that generates weekly performance summaries and health insights.

1.2 Purpose of Development

The beta testing phase is designed to validate that TrackMate effectively fulfills its objective of providing an integrated fitness and health management platform that is efficient, intuitive, and reliable before public deployment. In particular, we will:

1. **Verify Core Functionalities:**
   Ensure that key features such as workout logging, calorie and nutrition tracking, step counting using device sensors, trainer–student interaction, and AI-generated performance insights operate correctly and seamlessly.
2. **Evaluate User Experience & Usability:**
   Identify areas where users may face confusion in navigation or feature accessibility, ensuring the interface remains intuitive and user-friendly.
3. **Detect Functional and Integration Bugs:**
   Uncover issues in data synchronization, API interactions, authentication (including OTP verification), and real-time features such as notifications and tracking systems.
4. **Assess Performance and System Reliability:**
   Measure responsiveness, data processing efficiency, and stability across supported mobile devices, especially under realistic usage scenarios involving continuous tracking and background processes.
5. **Gather Feedback for Improvement:**
   Collect insights from testers regarding feature usefulness, AI-generated summaries, social and trainer features, and overall satisfaction to guide refinements before final release.

2 List of Reported Bugs

BUG 1:

Tested Feature: Incorrect Fallback Icons for Missing Exercise Assets

Tester Name: Mayukh Chowdhury

Testing Date:08/04/2026

Bug Details: During testing, it was observed that the same icon (the "running" icon, Icons.directions_run) is incorrectly being used for multiple distinct exercises (e.g., Running, Burpees, Lunges, etc.) across both the Exercise and Cardio sections of the application.

This creates confusion as users expect distinct icons or images for different activities, but instead view a uniform running icon regardless of the exercise selected.

Steps to Reproduce

1. Open the TrackMate app and navigate to the Exercise or Cardio Timer section.
2. Select any non-gym exercise such as "Burpee", "Lunges", or "Running".
3. Observe the icon displayed next to the exercise name or above the timer.
4. Expected Behavior: The app should display a distinct image or relevant icon for each exercise type.
5. Actual Behavior: The app falls back to displaying the Icons.directions_run (running) icon for all of them.

Bug Report Date:09/04/2026

Has the bug been fixed? Partially, as according to them: Since the proportion of images and the specificity was difficult to configure, we have applied a simple logo to everything and not changed any icon.

Date of Bug fixing:15/04/2026

Any other comment:NA

BUG 2:

Tested Feature: Incorrect Calorie Burn Calculation Logic

Tester Name: Mayukh Chowdhury

Testing Date:08/04/2026

Bug Details: There is a significant issue with how the application calculates calories burned during both Gym Sessions and Cardio exercises. The current calculation leads to highly inaccurate, vastly inflated results because it implements a flawed mathematical formula and fails to account for per-exercise multipliers during gym sessions.

Steps to Reproduce:

1. Open the TrackMate app and navigate to Exercise.
2. Select Cardio and run the timer for an exercise (e.g., Running at multiplier 4.8) for 30 minutes with a user weight of 70kg.
3. Observe the calculated calories burned upon finishing the session.
4. Expected Behavior: Using the standard MET formula (METs * Weight in kg * Time in minutes * 0.0175) , the burned calories should be (4.8 * 70 * 30 * 0.0175) ≈ 176.4 kcal .
5. Actual Behavior: The app calculates it as (30 / 30.0) * 4.8 * 70 = 336.0 kcal , which is nearly double the actual caloric burn.

Bug Report Date:09/04/2026

Has the bug been fixed? Yes

Date of Bug fixing:12/04/2026

Any other comment:NA

BUG 3:

Tested Feature: Forgot Password functionality is missing entirely

Tester Name: Mayukh Chowdhury

Testing Date:08/04/2026

Bug Details: During testing, it was discovered that users who forget their password have no way to recover their accounts. The "Forgot Password" feature is currently entirely absent from both the user interface and the underlying server backend.

Steps to Reproduce
1. Open the TrackMate app and navigate to the Login screen.
2. Observe the fields available for entry (Email and Password).
3. Expected Behavior: There should be a "Forgot Password?" link or button below the password field to initiate a password recovery flow.
4. Actual Behavior: No such link exists. The user is stranded if they forget their password.

Bug Report Date:09/04/2026

Has the bug been fixed? Yes

Date of Bug fixing:12/04/2026

Any other comment:NA

BUG 4:

Tested Feature: Chat UI Perpetual "Online" Status

Tester Name: Virendra Kala

Testing Date:08/04/2026

Bug Details: During testing, it was observed that when actively messaging through the application, the Chat UI perpetually displays "Online" beneath the recipient's name, even if the other user's app is closed or they are logged out. By testing with two different accounts on the same device, it is evident that User B will see User A as "Online" the moment User B opens the chat page, completely disregarding User A's actual network state. This behavior is misleading and defeats the purpose of presence tracking.

Steps to Reproduce

1. Open the application and log in with two different user accounts (User A and User B).
2. From User B's account, navigate to the chat page with User A.
3. Close or log out User A's application.
4. Observe the status shown for User A in User B's chat interface.

Expected Behavior: The status should dynamically reflect the real-time presence of User A (Offline when disconnected).

Actual Behavior: User A is still shown as "Online" regardless of actual connection status.

Bug Report Date: 09/04/2026

Has the bug been fixed? Partially. Instead of fully implementing real-time presence tracking via /presence API and WebSocket events (user_online/user_offline), the frontend logic only changed the hardcoded text from "Online" to "Connected" based on local connected status. The indicator is still tied to the current user's WebSocket connection and does not reflect the actual presence of the other user.

Date of Bug fixing:14/04/2026

Any other comment: NA

BUG 5:

Tested Feature: Email Verification Architectural Denial of Service Flaw

Tester Name: Virendra Kala

Testing Date: 08/04/2026

Bug Details: During testing, it was discovered that the application suffers from an email verification architecture flaw that allows malicious actors (or users making typographical errors) to permanently block legitimate users from registering. Currently, when a user registers, their email address and account details are immediately stored in the primary users database table before OTP verification is completed. Because the database enforces email uniqueness, an unverified account reserves the email indefinitely.

Steps to Reproduce

1. Attempt to register using an email address.
2. Do not complete the OTP verification process.
3. Try to register again using the same email address.

Expected Behavior: The system should allow re-registration or verification retry for unverified accounts.

Actual Behavior: The system blocks the email due to uniqueness constraints, preventing legitimate registration.

Bug Report Date: 09/04/2026

Has the bug been fixed? Yes. The backend logic (auth_service.py) was modified using the "Unverified Overwrite Approach". The system now checks if an existing email belongs to an unverified user and overwrites the existing record (updating password, name, new OTP, etc.), allowing successful registration without a 409 ConflictError.

Date of Bug fixing: 14/04/2026

Any other comment: NA

BUG 6:

Tested Feature: Missing Hourly Rate Pricing on Trainer Profiles

Tester Name: Virendra Kala

Testing Date:08/04/2026

Bug Details: During testing, it was observed that when users browse or search for trainers to book a session, the hourly rate (pricing) is missing from both the trainer profile and listing cards. This pricing information is essential for users to make informed decisions. Without visibility of cost per hour, users cannot evaluate affordability, which introduces friction in the booking process and may reduce conversions.

Steps to Reproduce

1. Open the application and navigate to the "Find Trainer" section.
2. Browse through available trainers.
3. Open any trainer profile.

Expected Behavior: Each trainer profile and listing card should display the hourly rate (e.g., ₹X/hr).

Actual Behavior: No pricing information is displayed.

Bug Report Date:09/04/2026

Has the bug been fixed? Yes. The hourly_rate field is now correctly serialized from the backend, and the UI has been updated in find_trainer_page.dart. Trainer listing cards and profile views now display the rate (e.g., "₹[Rate]/hr") or a fallback "Contact" if not configured.

Date of Bug fixing:14/04/2026

Any other comment: NA

BUG 7:

Tested Feature: Negative Values accepted for Experience and Hourly Rate

Tester Name: Praveen Baloda

Testing Date: :08/04/2026

Bug Details: During testing, it was observed that when submitting a Trainer Application, users are able to input negative numbers for their "Years of Experience" and "Hourly Rate" fields. The system accepts these inputs and persists them into the database without throwing any validation errors. This allows a trainer to have -5 years of experience or an hourly rate of ₹-500.

Steps to Reproduce

1. Open the Trainer Application form.
2. Enter a negative value in "Years of Experience" (e.g., -5).
3. Enter a negative value in "Hourly Rate" (e.g., -500).
4. Submit the application.

Expected Behavior: The system should reject negative values and display validation errors to the user.

Actual Behavior: The system accepts negative values and stores them in the database.

Bug Report Date::09/04/2026

Has the bug been fixed? Yes. Pydantic validators were implemented in backend/app/schemas/profile.py to reject negative values, and client-side checks (v < 0) were added in settings_page.dart using TextFormField validation. Data integrity is now ensured.

Date of Bug fixing: 14/04/26

Any other comment: NA

BUG 8:

Tested Feature: Missing Validation Feedback on Empty Text Submissions

Tester Name: Praveen Baloda

Testing Date: :08/04/2026

Bug Details: During testing, it was observed that when a user attempts to submit a "Trainer Request" or a "Social Post" with an empty text field, the application silently blocks the submission but does not provide any feedback. This leads to a confusing user experience where the user assumes the app is unresponsive or broken.

Steps to Reproduce

1. Navigate to the Trainer Request or Social Post section.
2. Leave the text input field empty.
3. Click the submit button.

Expected Behavior: The system should display an error message indicating that the field cannot be empty.

Actual Behavior: Nothing happens; no feedback or message is shown to the user.

Bug Report Date: :09/04/2026

Has the bug been fixed? No. The frontend logic in find_trainer_page.dart (Line 113) still uses if (goalCtrl.text.trim().isEmpty) return;, resulting in silent failure without any ScaffoldMessenger or user notification.

Date of Bug fixing: 14/04/26

Any other comment: NA

BUG 9:

Tested Feature: Global Cache Bleeding Across User Logins

Tester Name: Praveen Baloda

Testing Date: :08/04/2026

Bug Details: During testing, it was found that when multiple users log into the application on the same device, local cache data persists across accounts. For example, if User A submits a Trainer Application and logs out, then User B logs in, the app incorrectly shows that User B also has a pending Trainer Application.

Steps to Reproduce

1. Log in as User A.
2. Submit a Trainer Application.
3. Log out of User A's account.
4. Log in as User B on the same device.

Expected Behavior: User B should see only their own data, with no data carried over from User A.

Actual Behavior: User B sees cached data from User A, such as a pending Trainer Application.

Bug Report Date: :09/04/2026

Has the bug been fixed? Yes. The logout() function in auth_remote_datasource.dart was updated to clear cached preferences (prefs.remove('trainer_application')), ensuring proper session isolation.

Date of Bug fixing: 13/04/26

Any other comment: NA

BUG 10:

Tested Feature: Future Sessions Persistence After Trainer Quitting

Tester Name: Praveen Baloda

Testing Date::08/04/2026

Bug Details: During testing, it was observed that when a trainee or trainer ends their connection, any future sessions scheduled between them remain active and visible on both dashboards. This creates inconsistency and confusion, as sessions should no longer exist after the relationship ends.

Steps to Reproduce

1. Schedule future sessions between a trainer and a trainee.
2. Terminate the trainer-trainee connection.
3. Check the dashboard for upcoming sessions.

Expected Behavior: All future sessions between the two users should be removed or cancelled.

Actual Behavior: Future sessions remain visible and active.

Bug Report Date: :09/04/2026

Has the bug been fixed? Yes. The quit_trainer() function in trainer_service.py was updated with a database cascade hook to delete all future sessions (scheduled_at > time.now()). A system message is also triggered in the chat.

Date of Bug fixing: 15/04/26

Any other comment: NA

BUG 11:

Tested Feature: Chat Message Send Delay & Dispatch Update

Tester Name: Praveen Baloda

Testing Date: :08/04/2026

Bug Details: During testing, it was observed that when a user sends a message in the chat, the message does not immediately appear on the screen. The user must navigate away and return to the chat to see the message, creating the impression that the message was not sent.

Steps to Reproduce

1. Open a chat conversation.
2. Send a message.
3. Observe the chat screen immediately after sending.

Expected Behavior: The message should instantly appear in the chat interface.

Actual Behavior: The message does not appear until the chat is refreshed manually.

Bug Report Date: :09/04/2026

Has the bug been fixed? Yes. In chat_page.dart, the _send() function now uses optimistic UI updates by inserting a tempMessage into _messages using setState() and calling _scrollToBottom(), ensuring immediate visual feedback before server acknowledgment.

Date of Bug fixing: 17/04/26

Any other comment: NA

BUG 12:

Tested Feature: Negative Bounds on Gym Session Logging

Tester Name: Shreshthraj

Testing Date::08/04/2026

Bug Details: During testing, it was observed that while tracking a "Gym Session", users are able to input negatively signed values (e.g., -10 reps, -50 kg) and extremely large unrealistic values (e.g., 99999 reps, 5000 kg) into the fields for Reps, Weight, and Duration. The application accepts these invalid inputs without validation and sends them to the backend, which stores them in the database.

Steps to Reproduce

1. Open the Gym Session tracking interface.
2. Enter negative values (e.g., -10 reps, -50 kg).
3. Enter extremely large values (e.g., 99999 reps, 5000 kg).
4. Submit the set.

Expected Behavior: The system should validate input values, rejecting negative and unrealistic bounds with appropriate error messages.

Actual Behavior: The system accepts invalid values and stores them in the database.

Bug Report Date: :09/04/2026

Has the bug been fixed? Yes. Explicit bounds (ge=0, le=2000) were introduced via Pedantic Field validations in the LogSetRequest backend schema, and corresponding < 0 checks were added in the frontend _logSet() logic within exercise_page.dart.

Date of Bug fixing: 14/04/26

Any other comment: NA

BUG 13:

Tested Feature: Certifications Document Upload / Proof Verification

Tester Name: Shreshthraj

Testing Date: :08/04/2026

Bug Details: During testing, it was found that users applying to become Trainers can only enter certifications as comma-separated text. There is no provision to upload supporting documents or proof (e.g., certificates, PDFs, images), which limits verification and authenticity of credentials.

Steps to Reproduce

1. Navigate to the Trainer Application section.
2. Locate the "Certifications" input field.
3. Attempt to upload a document or file.

Expected Behavior: The system should allow users to upload certification documents (PDF/image) for verification.

Actual Behavior: Only text input is allowed; no file upload functionality exists.

Bug Report Date: :09/04/2026

Has the bug been fixed? No. The frontend (settings_page.dart) and backend (trainer.py) still rely on an Optional[str] comma-separated format. No multipart/form-data handling or cloud storage integration has been implemented.

Date of Bug fixing: 15/04/26

Any other comment: However the developer team mentioned that they did not add this feature , as this feature require significant changes and work which they were unable to do at that time .

BUG 14:

Tested Feature: Gym Session Set Data Loss (Hardcoded UUID)

Tester Name: Shreshthraj

Testing Date:08/04/2026

Bug Details: During testing, it was observed that when a trainee logs a "Gym Session" and records multiple sets, the sets appear correctly during the session. However, after finishing the workout, both Trainer and Trainee are unable to view the sets in history. The UI displays the session title with "0 sets", caused by a backend issue related to a hardcoded UUID mapping failure.

Steps to Reproduce

1. Start a Gym Session.
2. Log multiple exercise sets.
3. Complete the session.
4. Navigate to workout history.

Expected Behavior: All logged sets should be visible in workout history.

Actual Behavior: The session appears with "0 sets" and data is lost.

Bug Report Date::09/04/2026

Has the bug been fixed? Yes. The _logSet() payload in exercise_page.dart was corrected to pass the dynamic _selectedExercise![['id']]. Additionally, the silent catch block was removed, ensuring failed API calls roll back local state and trigger a ScaffoldMessenger warning.

Date of Bug fixing: 15/04/26

Any other comment: NA

BUG 15:

Tested Feature: Scheduling Sessions in the Past

Tester Name: shreshthraj

Testing Date:08/04/26

Bug Details: During testing, it was observed that while creating or scheduling a new session, trainers are able to select dates and times that have already elapsed (e.g., yesterday or earlier the same day). The application accepts these past timestamps without displaying any validation error and successfully stores them in the database. This leads to an inconsistent schedule where sessions instantly appear as "pending" despite already being in the past.

Steps to Reproduce

1. Navigate to the session scheduling interface.
2. Select a date and time that has already passed.
3. Confirm and create the session.
4. Check the dashboard or session list.

Expected Behavior:
The system should prevent scheduling sessions in the past and display an error message if an invalid timestamp is selected. Validation should exist on both frontend and backend.

Actual Behavior:
The frontend restricts past dates, but backend allows past timestamps via API, resulting in invalid sessions being created.

Bug Report Date: 09/04/26

Has the bug been fixed? Partially.

Frontend: Fixed. In student_detail_page.dart, the showDatePicker enforces firstDate: now, preventing selection of past dates. Additionally, a .isBefore(now) check ensures that invalid timestamps trigger a red SnackBar warning.

Backend: Not Fixed. In trainer.py, the ScheduleSessionRequest schema lacks a Pydantic @field_validator to enforce that scheduled_at must be greater than datetime.now(timezone.utc). This allows users to bypass frontend checks via direct API calls and create invalid sessions.

Date of Bug fixing: 14/04/26

Any other comment: Client-side functionality works correctly, but backend validation is still required for a complete and secure fix.

BUG 16:

Tested Feature: Session Status Transmissions and Dashboard Clearing

Tester Name: Virendra Kala

Testing Date:08/04/2026

Bug Details: During testing, it was observed that when a scheduled session reaches its start time or has entirely elapsed, it remains stuck in the "Upcoming Sessions" list for both the Trainer and the Trainee. Users do not have any UI option (such as a "Finish Session" or "Mark as Done" button) to manually complete the session. Additionally, the system does not automatically update the session status based on the current time. This leads to past and ongoing sessions cluttering the upcoming sessions view, causing confusion and affecting scheduling accuracy.

Steps to Reproduce

1. Schedule a session between a trainer and a trainee.
2. Wait until the session start time passes or the session ends.
3. Open the dashboard for either user.
4. Observe the "Upcoming Sessions" section.

Expected Behavior:
Sessions should automatically transition between "Upcoming", "Ongoing", and "Past" based on time. Past sessions should not appear in the "Upcoming Sessions" list, and users should have an option to mark sessions as completed.

Actual Behavior:
Sessions remain in the "Upcoming Sessions" list regardless of time, with no manual completion option available.

Bug Report Date:09/04/2026

Has the bug been fixed? Yes. The frontend logic (e.g., in trainer_requests_page.dart) was updated to dynamically categorize sessions into "Ongoing", "Upcoming", and "Past" using time-based comparisons (DateTime.now()). This resolves the cluttering issue from the user interface perspective.

Date of Bug fixing: 14/04/26

Any other comment: Visually on the client side, it functions well and cleanly separates the lists as requested!

3 Overall Quality of the Software

1. **How good is the user manual? Was it easy to install and run the software?**

   The user manual is clear, structured, and easy to follow, especially for end users. It provides step-by-step guidance for installation (APK setup), account creation with OTP verification, and navigation across major features such as dashboard, food logging, workouts, analytics, and social interactions. The inclusion of screenshots and detailed explanations of each screen significantly improves usability and onboarding.

   Installation is straightforward for Android users, with clearly defined steps for enabling APK installation and launching the app. However, the manual is more user-focused and lacks depth for developers—particularly regarding backend setup, API configuration, and deployment steps. Including a troubleshooting section for installation issues and clearer system dependency notes would further improve it.

2. **How is the quality of the code? Is it well documented? Is the code modular? Are the variables and the function names meaningful?**

   The codebase demonstrates strong architectural design and modularity. The project follows a clean separation between frontend (Flutter) and backend (FastAPI), with a layered structure and feature-first organization in the mobile app. The use of BLoC architecture, repository patterns, and service layers indicates adherence to good software engineering practices.

   Variable and function naming appear meaningful and consistent with modern conventions, improving readability and maintainability. The use of tools like SQLAlchemy, Alembic, and structured API endpoints further reflects a well-organized backend.

   However, documentation is somewhat limited at the code level. While high-level structure is clear, some complex logic (especially asynchronous workflows and integrations) would benefit from more inline comments and developer-focused documentation. Additionally, explicit mention of testing coverage is limited, suggesting room for improvement in test completeness and robustness.

3. **Do you think that the software lacks some major features that may make it less acceptable to the user?**

   The application provides a comprehensive and well-integrated set of features, including workout tracking, nutrition management, step counting, trainer–student

interaction, social engagement, and AI-generated performance insights. This makes it a strong all-in-one fitness platform.

However, a few enhancements could improve user acceptance:

- Limited offline functionality may restrict usability in low-connectivity environments.
- More personalization options (custom goals, adaptive recommendations) could enhance engagement.
- Broader integration with external fitness devices or apps (e.g., wearables) would improve tracking accuracy.
- Advanced analytics or deeper AI insights could make the platform more competitive.

4. **Does the software not satisfy any major non-functional requirements?**

The system meets most key non-functional requirements, including performance, modularity, security (JWT-based authentication, encryption), and scalability considerations through technologies like FastAPI, PostgreSQL, and Redis.

That said, there are areas that need improvement:

- **Scalability:** Real-world load testing and handling of high concurrent users are not clearly validated.
- **Reliability:** Dependence on external APIs (LLM, nutrition databases) may introduce latency or failure points.
- **Offline Support:** The app heavily relies on network connectivity for core features.
- **Latency:** AI-generated insights and API responses may introduce delays depending on network conditions.

Overall Satisfaction:

Overall satisfaction with TrackMate is high. The application successfully delivers a well-integrated and feature-rich fitness tracking platform with a strong technical foundation and intuitive user experience. Its combination of health tracking, social features, and AI-driven insights makes it both functional and engaging.

While there are areas for improvement—particularly in scalability, documentation, and advanced feature depth—the application in its current state is stable, usable, and ready for beta-level deployment. With iterative refinements, it has strong potential for wider adoption.

Appendix A - Group Log

During beta testing, we maintained constant communication via our WhatsApp group to coordinate efforts and address issues promptly.

Meeting Time	Agendas
7 April 2025 (19:00 – 20:00 IST)	Setup the software in the system
8 April 2025 - 11 April 2025	Testing of functionalities and error handling
16 April 2025 (20:00 – 23:00 IST)	Testing after Group 4 debugged the code
14 April 2025 (18:00 – 19:30 IST)	Start writing beta testing
17 April 2025 (15:00 – 18:00 IST)	Finalize beta testing document